UPA TLAUTLA
UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

ACTIVIDAD PARCIAL 2

Algoritmos de búsqueda ciega (BFS / DFS)

Asignatura	Fundamentos de Inteligencia Artificial
Docente	Mtro. Javier García Lara
Grupo / Cuatrimestre	171
Alumno(a)	Hidalgo López Karina Paola
Matrícula	24305030
Tema	Código + Comparativa: BFS y DFS resolviendo un laberinto
Fecha de entrega	25 de octubre de 2026

1. Planteamiento del problema

Se tiene un laberinto en forma de cuadrícula de 21 × 21 celdas, donde cada celda es un pasillo o una pared. Un agente inicia en la entrada **S = (1,1)** y debe llegar a la salida **G = (19,19)** moviéndose únicamente hacia arriba, derecha, abajo o izquierda por celdas libres. El objetivo es resolverlo con dos algoritmos de búsqueda ciega, **Búsqueda en Anchura (BFS)** y **Búsqueda en Profundidad (DFS)**, y compararlos en tiempo, nodos expandidos, longitud de la solución y uso de memoria.

Datos de entrada

Elemento	Valor
Tamaño de la cuadrícula	21 × 21 (backtracking recursivo, semilla = 7)
Ciclos	Se abre 8 % de las paredes internas para crear caminos alternos
Nodos (celdas libres) |V|	208
Aristas |E|	218
Inicio / Meta	(1,1) / (19,19)
Orden de exploración de vecinos	arriba, derecha, abajo, izquierda

2. Formulación como problema de búsqueda

Componente	Definición
Estado	Celda (fila, columna) en la que se encuentra el agente
Estado inicial	S = (1,1)
Acciones	ARRIBA, DERECHA, ABAJO, IZQUIERDA (si la celda destino no es pared)
Modelo de transición	Resultado(s, a) = celda vecina en la dirección a
Prueba de meta	¿estado = (19,19)?
Costo de camino	1 por movimiento (grafo no ponderado)

Representación del grafo

El espacio de estados se representa como un **grafo no dirigido** mediante una **lista de adyacencia** (diccionario de Python: cada celda libre → lista de celdas vecinas libres). Esta estructura ocupa O(|V| + |E|) de memoria y permite obtener los sucesores de un nodo en tiempo O(grado). Se usa un diccionario de *padres* para evitar ciclos (conjunto de visitados) y reconstruir el camino solución.

3. Procedimiento

BFS usa una **cola FIFO** como frontera: expande primero todos los nodos de profundidad d antes de los de profundidad d+1. **DFS** usa una **pila LIFO** (versión iterativa): expande siempre el nodo más profundo de la frontera y retrocede (backtracking) al llegar a un callejón sin salida. Ambos llevan control de nodos visitados (búsqueda sobre grafo). Para cada algoritmo se registran: tiempo (promedio de 200 corridas), nodos expandidos, longitud del camino y tamaño máximo de la frontera. Además, para generalizar, se repitió el experimento en 40 laberintos aleatorios por cada tamaño (11, 21, 31 y 41).

4. Análisis de complejidad

Sea **b** el factor de ramificación, **d** la profundidad de la meta más superficial y **m** la profundidad máxima del espacio.

Propiedad	BFS	DFS
Estructura de frontera	Cola (FIFO)	Pila (LIFO)
Completitud	Sí (b finito)	Sí en grafos finitos con control de visitados; No en árboles infinitos o sin control de ciclos
Optimalidad	Sí, si el costo de paso es uniforme (aquí 1)	No
Complejidad temporal	O(b^d) en árbol; O(|V|+|E|) en grafo	O(b^m) en árbol; O(|V|+|E|) en grafo
Complejidad espacial	O(b^d): guarda toda la frontera	O(b·m) en árbol; O(|V|) en grafo con visitados

Nota: en un laberinto b es pequeño (grado promedio ≈ 2.1), por eso la ventaja teórica de memoria de DFS no se observa de forma clara.

5. Resultados

5.1 Laberinto principal (21 × 21)

Algoritmo	Tiempo (ms)	Nodos expandidos	Longitud del camino	Máx. frontera
BFS	0.053	129	60	5
DFS	0.032	63	62	9

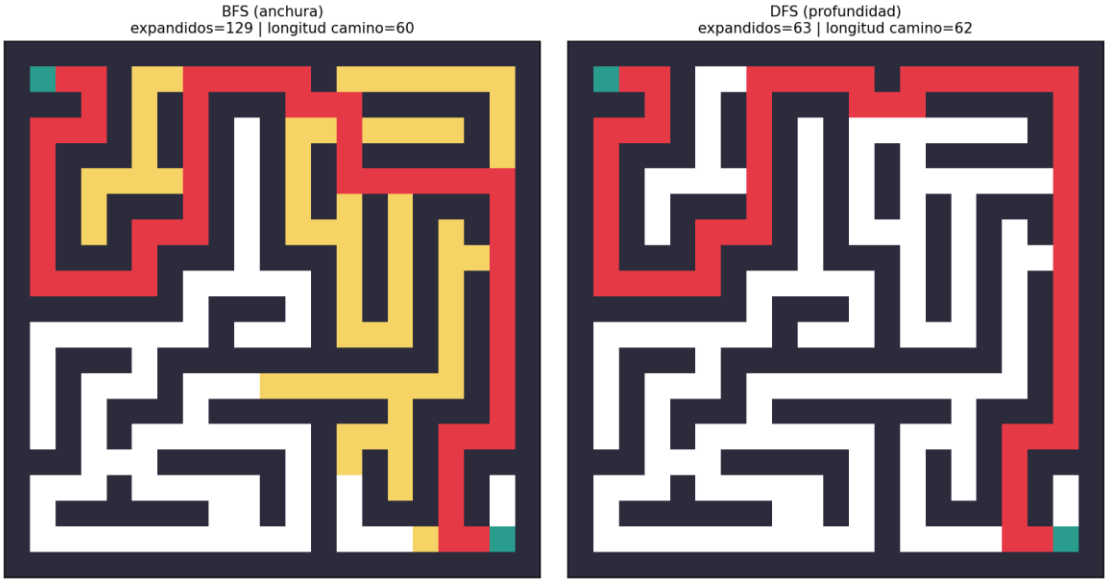


Figura 1. Verde: inicio y meta · Rojo: camino solución · Amarillo: nodos explorados · Blanco: no visitados.

5.2 Promedios en 40 laberintos aleatorios por tamaño

Tamaño	Expand. BFS	Expand. DFS	Camino BFS	Camino DFS	Frontera BFS	Frontera DFS
11 × 11	35.8	27.8	20.6	22.8	3.8	4.0
21 × 21	167.9	98.0	50.4	72.8	8.3	11.6
31 × 31	431.6	156.9	76.6	120.5	14.2	18.9
41 × 41	782.5	302.0	112.0	190.4	19.2	30.8

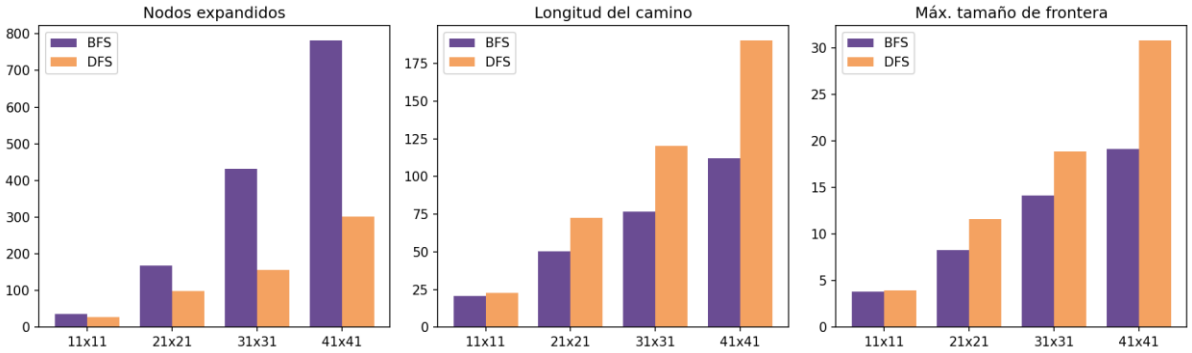


Figura 2. Comparación de métricas promedio entre BFS y DFS al crecer el laberinto.

6. Código Python

El código completo y comentado está en el archivo busqueda_ciega_laberinto.py del repositorio. Se muestran las partes esenciales.

Construcción del grafo

```
ORDEN = [(-1, 0), (0, 1), (1, 0), (0, -1)]  # arriba, derecha, abajo, izquierda



def construir_grafo(m):
    """Cada celda libre es un nodo; hay arista entre celdas libres adyacentes."""
    grafo = {}
    filas, cols = m.shape
    for f in range(filas):
        for c in range(cols):
            if m[f, c] == 0:
                grafo[(f, c)] = [(f + df, c + dc) for df, dc in ORDEN
                                 if 0 <= f + df < filas and 0 <= c + dc < cols
                                 and m[f + df, c + dc] == 0]
    return grafo
```

BFS y DFS

```
def reconstruir(padres, meta):
    camino, nodo = [], meta
    while nodo is not None:
        camino.append(nodo)
        nodo = padres[nodo]
    return camino[::-1]



def bfs(grafo, inicio, meta):
    """Búsqueda en anchura: frontera FIFO (cola)."""
    t0 = time.perf_counter()
    frontera = deque([inicio])
    padres = {inicio: None}          # sirve también como conjunto de visitados
    expandidos, orden, max_front = 0, [], 1
    while frontera:
        nodo = frontera.popleft()
        expandidos += 1
        orden.append(nodo)
        if nodo == meta:
            return dict(camino=reconstruir(padres, meta), expandidos=expandidos,
                        max_frontera=max_front, orden=orden,
                        tiempo=time.perf_counter() - t0)
        for v in grafo[nodo]:
            if v not in padres:
                padres[v] = nodo
                frontera.append(v)
        max_front = max(max_front, len(frontera))
    return dict(camino=None, expandidos=expandidos, max_frontera=max_front,
                orden=orden, tiempo=time.perf_counter() - t0)



def dfs(grafo, inicio, meta):
    """Búsqueda en profundidad: frontera LIFO (pila), versión iterativa."""
    t0 = time.perf_counter()
    frontera = [inicio]
    padres = {inicio: None}
    visitados = set()
    expandidos, orden, max_front = 0, [], 1
    while frontera:
        nodo = frontera.pop()
        if nodo in visitados:
            continue
        visitados.add(nodo)
        expandidos += 1
        orden.append(nodo)
        if nodo == meta:
            return dict(camino=reconstruir(padres, meta), expandidos=expandidos,
                        max_frontera=max_front, orden=orden,
                        tiempo=time.perf_counter() - t0)
        for v in reversed(grafo[nodo]):   # reversed para explorar en el mismo orden que BFS
            if v not in visitados:
                padres[v] = nodo
                frontera.append(v)
        max_front = max(max_front, len(frontera))
    return dict(camino=None, expandidos=expandidos, max_frontera=max_front,
                orden=orden, tiempo=time.perf_counter() - t0)
```

7. Análisis y conclusiones

Optimalidad. BFS siempre devolvió el camino más corto (60 pasos en el laberinto principal), como predice la teoría para costos uniformes. DFS encontró caminos válidos pero más largos (62 pasos en el laberinto principal y, en promedio, entre 1.1 y 1.7 veces más largos en los 40 laberintos por tamaño), porque devuelve el primer camino que encuentra y no el mejor.

Eficiencia (nodos expandidos y tiempo). En estos laberintos DFS expandió menos nodos (63 vs. 129 en el principal; 302 vs. 782 en 41 × 41) y fue más rápido. Esto se debe a que la meta está lejos de la entrada (esquina opuesta): BFS debe explorar todas las celdas con profundidad menor a la de la meta, mientras que DFS avanza en línea y, con suerte, no se desvía mucho. Es un resultado dependiente del problema: si la meta estuviera cerca de S, BFS sería mejor, y DFS puede tener un mal caso si elige primero una rama larga equivocada.

Memoria. Contrario a lo esperado en teoría, la frontera máxima fue menor en BFS (5 vs. 9; 19.2 vs. 30.8 en 41 × 41). Con la versión iterativa de DFS que se implementó, se apilan todos los sucesores no visitados, y en laberintos con ciclos esto acumula entradas repetidas; además, el factor de ramificación del laberinto es muy bajo. En otros espacios con b grande (por ejemplo, un puzzle de 8 piezas) la ventaja de memoria de DFS sí sería evidente.

¿Cuál es más idóneo?

Situación	Algoritmo recomendado
Se requiere el camino más corto (costo uniforme)	BFS
Meta superficial / cercana al inicio	BFS
Espacio muy grande, b alto y memoria limitada	DFS (o profundización iterativa)
Basta con encontrar *una* solución y hay muchas	DFS
Espacio infinito o con ciclos sin control de visitados	BFS (DFS puede no terminar)

Para el problema del laberinto, si importa la **calidad** de la solución (ruta mínima), BFS es la elección correcta; si solo interesa llegar rápido y el costo de explorar es lo crítico, DFS puede ser suficiente. Como ninguno usa información de la meta, ambos son superados por búsquedas heurísticas (A*), que se verán en la sección 2.5.

8. Declaración de uso de IA

Herramienta	Prompt	Uso	Verificación
Claude (Anthropic)	“Ayúdame a desarrollar la actividad 2.4” (con imagen del documento de la actividad)	Generación del código base, de las figuras y de la estructura del reporte	[Completar: el alumno ejecutó el código, revisó los resultados y entiende cada parte]

9. Referencias (APA 7.ª ed.)

Bohm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science.

García Serrano, A. (2016). *Inteligencia artificial: Fundamentos, práctica y aplicaciones* (2.ª ed.). RC Libros.

NetworkX Developers. (s. f.). *NetworkX documentation*. https://networkx.org

Python Software Foundation. (s. f.). *Python documentation*. https://www.python.org

Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4.ª ed.). Pearson.